

**POSTS AND TELECOMMUNICATIONS INSTITUTE OF
TECHNOLOGY**

CENTER FOR INTERNATIONAL EDUCATION



Advanced Information Security

Topic: Blockchain Technology and Applications

Lecturer: Assoc. Prof. Dr. Hoang Xuan Dau

Student ID: M25VMCS12

Date June 23, 2026

Abstract

This report studies blockchain technology from the viewpoint of Advanced Information Security. Blockchain is not simply a cryptocurrency technology; it is a distributed ledger architecture that combines cryptographic hash functions, digital signatures, peer-to-peer networking and consensus mechanisms to provide data integrity, authentication, non-repudiation, transparency and tamper resistance. The report first explains the fundamental structure of blockchain, including blocks, transactions, ledgers, nodes, smart contracts and different blockchain models. It then analyzes the security mechanisms behind blockchain and clarifies the difference between blockchain security and traditional encryption topics such as AES. The report also compares major platforms including Bitcoin, Ethereum and Hyperledger Fabric, discusses applications in finance, supply chain, digital identity, healthcare, certificate verification and audit logging, and evaluates common security risks such as 51% attacks, double spending, private-key theft, Sybil attacks, smart contract vulnerabilities and oracle problems. Finally, the report proposes an Ethereum local blockchain experiment using Hardhat, Solidity and a React frontend to demonstrate a blockchain-based supply chain tracking system, showing how product creation, status updates and integrity verification can be carried out through transaction hashes and blockchain state updates. The conclusion is that blockchain is highly relevant to information security, but it should be applied only when multiple parties need a shared, append-only and auditable source of truth.

Keywords

Blockchain, distributed ledger, cryptographic hash, digital signature, consensus, smart contract, Ethereum, Hyperledger Fabric, information security.

Table of contents

Abstract.....	2
Table of contents.....	3
List Of Abbreviations.....	5
1. Introduction / Overview.....	6
1.1. Assignment Context and Topic Scope.....	6
1.2. Background and Motivation.....	7
1.3. What Is Blockchain?.....	7
1.4. Why Blockchain Belongs to Advanced Information Security.....	8
1.5. Report Objectives.....	8
1.6. Research Methodology.....	9
2. Blockchain Fundamentals and Architecture.....	9
2.1. Distributed Ledger Technology.....	9
2.2. Blocks, Transactions and Chain Structure.....	9
2.3. Cryptographic Hash Functions in the Data Layer.....	10
2.4. Nodes and Peer-to-Peer Networking.....	11
2.5. Types of Blockchain.....	11
2.6. Transaction Lifecycle.....	12
2.7. Smart Contracts.....	12
3. Security Mechanisms in Blockchain.....	13
3.1. Hash Functions and Data Integrity.....	13
3.2. Public Keys, Private Keys and Digital Signatures.....	13
3.3. Consensus Mechanisms.....	14
3.3.1. Proof of Work.....	14
3.3.2 Proof of Stake.....	14
3.3.3 Byzantine Fault Tolerance and Permissioned Consensus.....	15
3.4. Identity and Access Control in Permissioned Blockchain.....	15
3.5. Security Properties Achieved by Blockchain.....	15
3.6. What Blockchain Does Not Automatically Solve.....	16
4. Blockchain Platforms and Applications.....	16
4.1. Bitcoin.....	16
4.2. Ethereum.....	16
4.3. Hyperledger Fabric.....	17
4.4. Platform Comparison.....	17
4.5. Applications of Blockchain.....	18
4.5.1. Cryptocurrency and Digital Payments.....	18
4.5.2. Supply Chain Management.....	18
4.5.3. Certificate Verification.....	18

4.5.4. Digital Identity.....	19
4.5.5. Healthcare Records.....	19
4.5.6. E-Voting.....	19
4.5.7. Audit Logging and Compliance.....	19
4.6. When Blockchain Is Not Needed.....	19
5. Security Analysis of Blockchain Systems.....	20
5.1. Threat Model.....	20
5.2. 51% Attack and Chain Reorganization.....	20
5.3. Double Spending.....	20
5.4. Sybil Attacks and Network Attacks.....	21
5.5. Private Key Theft.....	21
5.6. Smart Contract Vulnerabilities.....	21
5.7. Oracle Problem.....	22
5.8. Privacy and Compliance Risks.....	22
5.9. Governance and Upgrade Risks.....	23
5.10. Security Controls and Best Practices.....	23
6. Experiment / Demo: Blockchain-Based Supply Chain Tracking System.....	23
6.1. Demo Overview.....	23
6.2. Demo Objectives.....	24
6.3. Tools and Technologies Used.....	24
6.4. System Architecture.....	25
6.5. Smart Contract Design.....	26
6.6. Installation and Configuration.....	28
6.7. Demo Procedure.....	28
6.8. Demo Results.....	28
6.9. Security Explanation of the Demo.....	28
6.10. Evaluation.....	29
7. Advantages, Limitations and Future Trends.....	29
7.1. Advantages of Blockchain.....	29
7.2. Limitations of Blockchain.....	29
7.3 Future Trends.....	30
7.3.1. Zero-Knowledge Proofs.....	30
7.3.2. Decentralized Identity and Verifiable Credentials.....	30
7.3.3. Post-Quantum Cryptography.....	30
7.3.4. Enterprise and Government Use.....	30
8. Conclusion.....	31
Member Contribution Table.....	32
References.....	33

List Of Abbreviations

Abbreviation	Meaning
AES	Advanced Encryption Standard
API	Application Programming Interface
BFT	Byzantine Fault Tolerance
DLT	Distributed Ledger Technology
DApp	Decentralized Application
EVM	Ethereum Virtual Machine
IDS	Intrusion Detection System
MSP	Membership Service Provider
PBFT	Practical Byzantine Fault Tolerance
PKI	Public Key Infrastructure
PoS	Proof of Stake
PoW	Proof of Work
RSA	Rivest-Shamir-Adleman
SHA	Secure Hash Algorithm
TLS	Transport Layer Security
ZKP	Zero-Knowledge Proof

Table 1. Abbreviations used in the report

1. Introduction / Overview

1.1. Assignment Context and Topic Scope

The project brief for the Advanced Information Security course assigns Group 3 to research blockchain technology and applications, investigate several blockchain platforms, and set up an experimental blockchain platform. The required report structure includes an introduction or overview, main content covering architecture, features, advantages, disadvantages, installation, configuration and demo, followed by a conclusion and references [1]. Therefore, this report is written as a complete project report rather than as a short survey.

The selected topic is different from the AES topic in the same assignment. AES is a symmetric encryption algorithm mainly focused on confidentiality, encryption rounds, key scheduling, attacks and defenses. Blockchain, by contrast, is a distributed system architecture. It uses cryptography, but its main security goals are integrity, authenticity, non-repudiation, auditability and decentralized trust. In other words, blockchain is related to cryptography, but it should not be presented as another encryption algorithm.

Question	AES Topic	Blockchain Topic
Main focus	A single symmetric encryption algorithm	A distributed ledger architecture and its applications
Primary security goal	Confidentiality of data	Integrity, authenticity, non-repudiation, transparency and tamper resistance
Typical techniques	Substitution-permutation network, round keys, encryption/decryption	Hash-linked blocks, digital signatures, consensus, peer-to-peer replication
Demo direction	Encrypt and decrypt a message	Create transactions, deploy a smart contract, verify hashes and ledger state
Risk of overlap	Low if AES is explained deeply	Low if cryptography is explained only as a blockchain component

Table 2. Difference between the AES project and the blockchain project

1.2. Background and Motivation

Modern organizations frequently need to share records across boundaries. Banks exchange payment records, hospitals share patient histories, universities verify certificates, logistics companies track goods, and government agencies maintain public registries. In a traditional system, one central database owner is responsible for storing and validating records. This design is efficient, but it creates a central point of trust and a central point of failure. If the central database is attacked, corrupted or manipulated by an insider, other participants may have difficulty proving what the correct historical record was.

Blockchain technology was introduced to address a specific trust problem: how can a community of participants maintain a shared transaction history without fully trusting a single central authority? The original Bitcoin paper proposed a peer-to-peer electronic cash system in which digital signatures are used for ownership and a peer-to-peer network prevents double spending by timestamping transactions into a chain of proof-of-work [3]. Since then, blockchain ideas have expanded beyond cryptocurrency into enterprise ledgers, smart contracts, digital identity, asset tracking, supply chain transparency and verifiable credentials.

From the perspective of information security, blockchain is important because it provides a practical architecture for tamper-evident record keeping. NIST describes blockchains as tamper-evident and tamper-resistant digital ledgers implemented in a distributed fashion, generally without a central repository and usually without a central authority [2]. This definition directly connects blockchain to security principles such as integrity, accountability, resilience and auditability.

1.3. What Is Blockchain?

A blockchain is a distributed ledger that stores records in a sequence of blocks. Each block contains a group of transactions or state changes. Each block also contains a cryptographic hash of the previous block. This previous-block hash creates a chain: if an attacker changes a past transaction, the hash of that block changes, which also breaks the link to every later block. Because copies of the ledger are stored by many network nodes, a malicious user cannot easily rewrite history unless the network consensus mechanism is also defeated.

The term blockchain is sometimes used loosely. In this report, blockchain means a system with four essential features: an append-only ledger, cryptographic linking of records, network replication among multiple participants, and a consensus or validation process for deciding which transactions are accepted. A normal database with a hash column is not necessarily a

blockchain. Likewise, a blockchain is not automatically secure if its keys, consensus rules, smart contracts or governance model are weak.

1.4. Why Blockchain Belongs to Advanced Information Security

Blockchain belongs to Advanced Information Security because it combines several advanced security concepts into a working distributed system. First, it uses hash functions to provide integrity and tamper detection. Second, it uses asymmetric cryptography and digital signatures to prove transaction ownership and authenticate senders. Third, it uses consensus protocols to defend the shared ledger against inconsistent or malicious updates. Fourth, in permissioned systems such as Hyperledger Fabric, identity, membership, access control and endorsement policies are used to decide who can join the network and who must approve a transaction [8], [9].

However, it is important to avoid a common misunderstanding: blockchain is not mainly an encryption technology. Public blockchains usually make transaction data visible to network participants. The security property is not secrecy by default; the main property is that the published record is difficult to modify without detection. Confidentiality, when required, must be added through encryption, private channels, off-chain storage, zero-knowledge proofs or permissioned access control.

1.5. Report Objectives

- Explain the main concepts, architecture and operation of blockchain technology.
- Analyze the role of hash functions, digital signatures and consensus in blockchain security.
- Compare Bitcoin, Ethereum and Hyperledger Fabric as representative blockchain platforms.
- Discuss practical applications and evaluate when blockchain is useful or unnecessary.
- Analyze key attacks, vulnerabilities and security controls for blockchain systems.
- Design a simple demo using Hardhat, Solidity and a local Ethereum blockchain.

1.6. Research Methodology

This report is based on official documentation and reputable security references. Key sources include the NIST Blockchain Technology Overview, the Bitcoin whitepaper, Ethereum developer documentation, Hyperledger Fabric documentation, Solidity documentation, OWASP Smart Contract Top 10, OpenZeppelin smart contract guidance and IBM materials on blockchain benefits. These sources are combined with technical analysis to build a report suitable for a course project in Advanced Information Security.

2. Blockchain Fundamentals and Architecture

2.1. Distributed Ledger Technology

A ledger is a record of facts and transactions. A bank ledger records balances and changes to balances. A supply chain ledger records product origins, custody transfers and status changes. Distributed Ledger Technology, or DLT, means that copies of a logically shared ledger are maintained across multiple nodes. The important idea is that participants can verify a common history without depending only on one database administrator.

In a blockchain-based distributed ledger, transactions are grouped into blocks, and blocks are appended to the ledger in a controlled order. Hyperledger Fabric documentation explains a similar idea by separating the ledger into a blockchain transaction log and a world state database: the blockchain stores the immutable history, while the world state stores current values for convenient application access [10]. Public blockchains and enterprise blockchains may implement this model differently, but the security principle is the same: a current state should be traceable back to an ordered, verifiable transaction history.

2.2. Blocks, Transactions and Chain Structure

A transaction is a request to change the ledger state. In a cryptocurrency system, a transaction may transfer coins from one address to another. In a certificate verification system, a transaction may register the hash of a certificate. In a supply chain system, a transaction may update an asset status from manufactured to shipped. Transactions are collected into blocks so that the system can validate and store them efficiently.

A typical block contains a block header and a list of transactions. The block header may include the previous block hash, timestamp, Merkle root, block number, nonce or validator signature depending on the platform. The previous block hash is the most important structural

field because it links a block to its parent. The Merkle root summarizes all transactions in the block and allows efficient verification that a transaction belongs to the block.

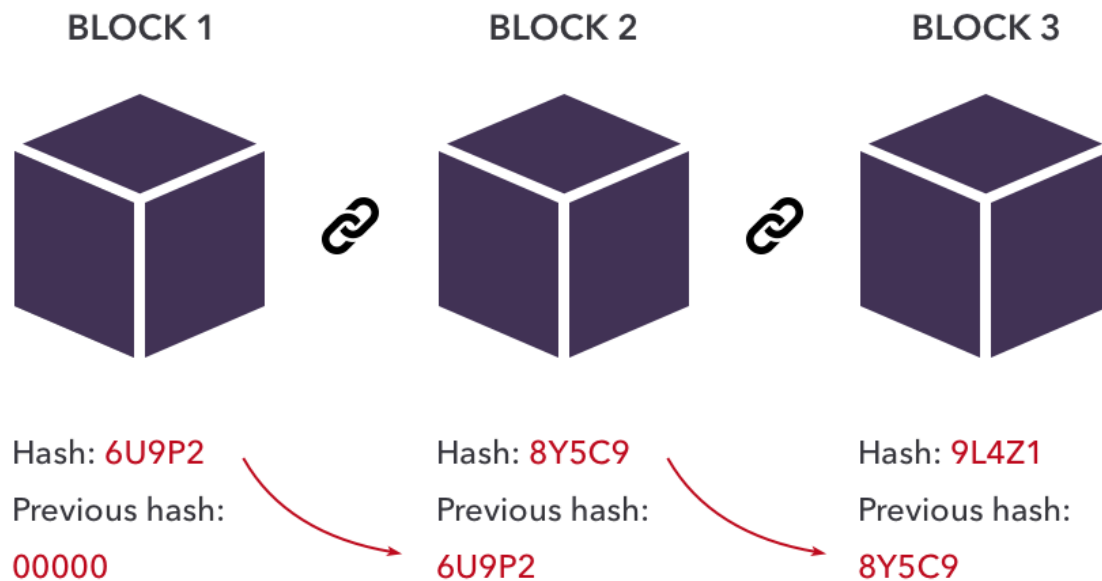


Figure 1. Simplified blockchain structure: each block stores the hash of the previous block, forming a tamper-evident chain.

If a transaction inside Block 1 is modified, the Merkle root of Block 1 changes. Then the hash of Block 1 changes. Because Block 2 stores the previous hash of Block 1, Block 2 no longer points to the correct value. This cascading effect is why blockchains are considered tamper-evident. Tampering is not impossible in theory, but it becomes visible and expensive to hide because the attacker must also recompute or reorganize following blocks and convince the network to accept the manipulated history.

2.3. Cryptographic Hash Functions in the Data Layer

A cryptographic hash function maps input data of arbitrary length to a fixed-length digest. A secure hash function should be one-way, collision-resistant and sensitive to small input changes. NIST notes that SHA-256 has a huge output space and is specified in FIPS 180-4 [2]. In blockchain systems, hash functions are used to identify blocks, link blocks, summarize transactions, build Merkle trees and support proof-of-work in some networks.

Hashing should not be confused with encryption. Encryption is reversible if the correct key is known. Hashing is designed to be non-reversible. In blockchain, the purpose of hashing is usually not to hide data but to create a compact fingerprint of data. If the data changes, the fingerprint changes. This makes hash functions ideal for integrity checking.

2.4. Nodes and Peer-to-Peer Networking

A blockchain network consists of nodes. A full node stores and verifies the ledger. A light node may store only a small part of the ledger and rely on full nodes for proofs. In proof-of-work systems, miners compete to create blocks. In proof-of-stake systems, validators are selected to propose or attest to blocks. In permissioned enterprise systems, peer nodes, ordering nodes and certificate authorities may perform separate functions.

The peer-to-peer network is important for availability and decentralization. When a transaction is created, it is broadcast to nearby nodes and propagated through the network. Nodes validate the transaction according to protocol rules. If valid, it is stored in a pending transaction pool until a block producer or ordering service includes it in a block. This distributed communication model reduces dependency on a single server, but it also introduces new risks such as network partitioning, eclipse attacks, denial of service and inconsistent views of the chain.

2.5. Types of Blockchain

Type	Description	Security and Governance Characteristics	Examples
Public blockchain	Anyone can read, submit transactions and usually run a node	High openness and transparency. Consensus must defend against unknown participants. Privacy is limited by default	Bitcoin, Ethereum
Private blockchain	Controlled by one organization or a limited administrative domain	Easier governance and access control, but less decentralized. Often resembles a secured distributed database	Internal enterprise ledger
Consortium / permissioned blockchain	Controlled by a group of known organizations	Suitable when several organizations need a shared ledger but do not want a fully public network	Hyperledger Fabric network among banks, logistics partners or hospitals

Table 3. Main types of blockchain networks

The correct blockchain type depends on the use case. Public blockchains are useful when censorship resistance and open participation are important. Permissioned blockchains are more suitable when participants are known, compliance matters, and transaction data should be restricted. For an information security course, comparing public and permissioned models is important because their threat models differ.

2.6. Transaction Lifecycle

1. A user or application creates a transaction request.
2. The transaction is signed using the private key of the sender or submitted using an authenticated organizational identity.
3. The transaction is broadcast to the network or sent to peers/orderers in a permissioned network.
4. Nodes verify the transaction format, signature, balance, permissions and smart contract rules.
5. The transaction is ordered with other transactions and placed into a block.
6. The block is validated through the consensus mechanism or endorsement policy.
7. The block is appended to the ledger and replicated among nodes.
8. The application reads the updated state from the blockchain or world state database.

Ethereum proof-of-stake documentation gives a similar end-to-end view: a user creates and signs a transaction, nodes verify it, it enters a local mempool, a validator proposes a block, and later the transaction can become finalized when consensus conditions are met [5]. This lifecycle shows why blockchain security is a combination of cryptographic authentication, protocol validation and distributed consensus.

2.7. Smart Contracts

A smart contract is program logic stored and executed on a blockchain. Ethereum documentation describes smart contracts as a type of Ethereum account that can receive transactions and run code as programmed [4]. Solidity documentation defines Solidity as an object-oriented high-level language for implementing smart contracts that govern account behavior within Ethereum state [13]. Smart contracts can automate rules such as token transfers, asset ownership changes, certificate registration, voting, escrow and access control.

Smart contracts make blockchain platforms more flexible, but they also increase security risk. A bug in a smart contract can be permanently deployed and repeatedly exploited. Therefore, secure smart contract development requires input validation, access control, testing, code

review, audits and well-known libraries. Solidity documentation explicitly warns that smart contracts have unique issues and recommends code review, testing, audits and correctness proofs before production deployment [13].

3. Security Mechanisms in Blockchain

3.1. Hash Functions and Data Integrity

Integrity means that data has not been altered in an unauthorized way. Blockchain provides integrity by hashing transactions and blocks. If any transaction changes, the corresponding Merkle root and block hash change. Because later blocks refer to earlier blocks by hash, old manipulation becomes visible. This is why a blockchain can be described as an append-only log: new records can be added, but old records should not be quietly rewritten.

In a course project, this is the most important way to explain hash functions. The report does not need to go into the mathematical details of SHA-256, Keccak or all possible collision attacks. Instead, it should explain how hash functions are used in the blockchain architecture. This avoids overlap with other cryptography topics while still showing why blockchain is security-related.

3.2. Public Keys, Private Keys and Digital Signatures

Digital signatures provide authentication and non-repudiation. In a public blockchain, a user controls an address by controlling the corresponding private key. When the user sends a transaction, the transaction is signed with the private key. Nodes can verify the signature using public-key information without learning the private key. Ethereum accounts are entities with an ETH balance that can send messages, and accounts may be user-controlled or deployed as smart contracts [6].

The Bitcoin whitepaper also emphasizes digital signatures as part of the electronic cash solution [3]. However, it states that digital signatures alone are not enough because a trusted party would still be needed to prevent double spending. Blockchain combines signatures with a distributed timestamping and consensus mechanism so that the network can decide which transaction history is valid.

Private key protection is therefore one of the most critical operational security requirements in blockchain. If a private key is stolen, the attacker can sign transactions as the legitimate user. Unlike password-based systems, many blockchain transactions cannot be reversed by a

central administrator. This makes hardware wallets, secure key storage, multi-signature controls and good backup practices extremely important.

3.3. Consensus Mechanisms

Consensus is the process by which distributed participants agree on the valid ledger state. Without consensus, different nodes might accept different transaction histories. A consensus mechanism must handle network delays, honest mistakes and malicious attempts to rewrite history. The best consensus design depends on whether participants are unknown and permissionless or known and permissioned.

3.3.1. Proof of Work

Proof of Work is the original consensus mechanism used by Bitcoin. Miners compete to solve a computational puzzle. The solution proves that they spent computational resources. The Bitcoin whitepaper explains that the network timestamps transactions by hashing them into an ongoing chain of hash-based proof-of-work and that changing a record would require redoing the proof-of-work [3]. The advantage of proof-of-work is that it is simple and battle-tested. The disadvantages are high energy cost, mining centralization risks and limited throughput.

3.3.2 Proof of Stake

Proof of Stake replaces energy expenditure with economic stake. Ethereum documentation states that validators stake ETH into a smart contract; validators then check blocks, propose blocks and can lose staked ETH if they act dishonestly [5]. Ethereum switched to proof-of-stake in 2022 because it is less energy-intensive and provides better support for scaling solutions [5]. Proof of Stake can improve energy efficiency, but it is more complex and requires careful design to handle long-range attacks, validator behavior and finality.

3.3.3 Byzantine Fault Tolerance and Permissioned Consensus

Permissioned blockchains often use Byzantine Fault Tolerant or ordering-based consensus mechanisms. Because participants are known, the system can use identity, voting and endorsement policies instead of expensive mining. Hyperledger Fabric is a modular enterprise-grade distributed ledger platform with interchangeable components such as consensus and membership services [11]. Its model separates transaction endorsement, ordering and validation, allowing organizations to define who must approve a transaction.

3.4. Identity and Access Control in Permissioned Blockchain

Public blockchains rely heavily on pseudonymous public-key addresses. Permissioned blockchains add formal identity. Hyperledger Fabric uses a Membership Service Provider, or MSP, to define trusted organizations and validate member identities. Fabric documentation explains that a private key signs a transaction, and the corresponding public key in the MSP verifies that the signature is valid without revealing the private key [8].

Endorsement policies further strengthen access control. Fabric documentation states that every chaincode has an endorsement policy specifying which peers on a channel must execute the chaincode and endorse the results for a transaction to be valid [9]. This is highly relevant to information security because it combines identity, authorization and distributed approval.

3.5. Security Properties Achieved by Blockchain

Security property	How blockchain supports it	Important limitation
Integrity	Hashes link blocks and summarize transactions.	Integrity depends on secure hash functions and correct validation rules
Authentication	Transactions are signed using private keys or organizational identities	If private keys are stolen, the attacker can authenticate as the owner
Non-repudiation	A valid signature provides evidence that a private key authorized a transaction	Legal attribution may still require identity binding outside the chain
Availability	Ledger copies are distributed across multiple nodes	Networks can still suffer DoS, partitioning or node centralization
Auditability	Append-only history supports tracing and verification	Public transparency may conflict with privacy requirements
Tamper resistance	Changing old records requires breaking hash links and defeating consensus	Small private networks may be easier to collude against

Table 4. Blockchain security properties and limitations

3.6. What Blockchain Does Not Automatically Solve

Blockchain does not automatically solve confidentiality. A public blockchain may reveal addresses, balances, transaction timing and smart contract interactions. Blockchain also does not guarantee that off-chain data is true. If a fake certificate hash is registered, the chain can prove that the fake hash was registered, but it cannot automatically prove that the original certificate was legitimate. This is the oracle and data-quality problem.

Blockchain also does not remove the need for secure software engineering. Wallets, APIs, smart contracts, nodes and user interfaces can still have vulnerabilities. Smart contract security is especially important because deployed code can hold real assets and may be difficult or impossible to patch.

4. Blockchain Platforms and Applications

4.1. Bitcoin

Bitcoin is the first widely deployed blockchain application. Its main purpose is peer-to-peer electronic cash. The Bitcoin design focuses on preventing double spending without a central financial institution. It uses digital signatures to transfer ownership and proof-of-work to order transactions in a public ledger [3]. Bitcoin has high historical importance and strong security experience, but it has limited smart contract flexibility compared with Ethereum and limited suitability for enterprise permissioned workflows.

4.2. Ethereum

Ethereum expands the blockchain idea from simple payment transfer to programmable state. Smart contracts on Ethereum are deployed to the network and execute when called by transactions [4]. Ethereum transactions cost gas, and Ethereum documentation explains that simple transfers require gas units and that contract interactions include fees [7]. Gas prevents abuse by requiring users to pay for computation and storage.

Ethereum is a strong choice for a student demo because it has mature tools such as Hardhat, Ganache, Remix, MetaMask, ethers.js and Solidity. Hardhat documentation describes Hardhat as a flexible and extensible development environment for writing, testing, debugging and deploying Ethereum smart contracts [12]. A local Ethereum network makes it possible to demonstrate transaction hashes, block numbers and smart contract state without using real cryptocurrency.

4.3. Hyperledger Fabric

Hyperledger Fabric is an enterprise-grade permissioned distributed ledger framework. The LF Decentralized Trust project page describes Fabric as a modular enterprise-grade distributed ledger platform with components such as consensus and membership services [11]. Fabric is suitable for business networks where organizations know each other but still need a shared, auditable ledger. Examples include supply chain partners, trade finance organizations, healthcare consortia and government registries.

Fabric has several concepts that are especially relevant to security: membership service providers, channels, chaincode, endorsement policies, ordering service, peers, certificates and world state. A channel can isolate data among a subset of participants. Endorsement policies define which organizations must approve a transaction before it updates the ledger. This makes Fabric a strong example of permissioned blockchain security.

4.4. Platform Comparison

Criteria	Bitcoin	Ethereum	Hyperledger fabric
Network Model	Public, permissionless	Public mainnet; private/local networks possible	Permissioned enterprise network
Main purpose	Peer-to-peer electronic cash	Smart contracts and decentralized applications	Multi-organization business ledger
Identity	Pseudonymous addresses	Externally owned accounts and contract accounts	Certificate-based organizational identities
Consensus	Proof of Work	Proof of Stake on mainnet; local dev networks use simulated consensus	Modular ordering and endorsement model
Smart contract support	Limited scripting	Strong support through EVM and Solidity/Vyper	Chaincode in Go, JavaScript or Java

Privacy by default	Low	Low on public networks	Higher through permissioning and channels
Best demo for class	Transaction and proof-of-work explanation	Smart contract deployment and transaction hash demo	Enterprise asset-transfer network demo

Table 5. Comparison of Bitcoin, Ethereum and Hyperledger Fabric

4.5. Applications of Blockchain

4.5.1. Cryptocurrency and Digital Payments

Cryptocurrency remains the most famous blockchain application. It uses blockchain to record ownership and transfer of digital assets. Digital signatures authorize spending, while consensus prevents double spending. This application is security-heavy because the ledger itself controls economic value.

4.5.2. Supply Chain Management

Supply chain management is a common enterprise blockchain use case. Multiple organizations need to record product origin, custody transfer, quality checks and delivery status. IBM summarizes blockchain benefits as increasing trust, security and transparency among organizations by improving traceability of data shared across a business network [15]. In practice, blockchain is useful when different parties need a shared audit trail and no single party should be able to rewrite records alone.

4.5.3. Certificate Verification

Universities and training centers can register certificate hashes on a blockchain. A verifier later computes the hash of a presented certificate and compares it with the registered hash. This does not require storing the full certificate on-chain. The blockchain proves that a specific hash was registered at a particular time by an authorized issuer. This is a good student demo because it clearly shows data integrity and tamper detection.

4.5.4. Digital Identity

Blockchain can support decentralized identity systems where users control identifiers and present verifiable credentials. The security goal is to reduce dependence on centralized

identity providers and allow credentials to be verified cryptographically. However, identity systems must still handle privacy, revocation, recovery and legal recognition.

4.5.5. Healthcare Records

Healthcare systems can use blockchain for audit trails and consent records. Sensitive medical data should usually remain off-chain in protected databases, while the blockchain stores hashes, permissions or access logs. This approach supports integrity and accountability without exposing confidential records to all participants.

4.5.6. E-Voting

Blockchain voting is attractive because it promises transparent counting and tamper-resistant records. However, e-voting is also one of the hardest use cases because it requires voter privacy, coercion resistance, secure identity, malware-resistant devices and reliable dispute resolution. Blockchain can help with auditability but cannot by itself solve all election security problems.

4.5.7. Audit Logging and Compliance

Organizations can use blockchain-like ledgers to record security events, approvals, changes and compliance evidence. A tamper-evident audit log is useful when administrators should not be able to silently delete or modify historical events. This is closely related to information security monitoring and accountability.

4.6. When Blockchain Is Not Needed

Blockchain is not always the best solution. If one trusted organization controls all data and there is no need for multi-party verification, a traditional database with strong access control, backups and audit logs may be simpler and faster. Blockchain should be considered when several independent parties need to share records, when no party should have unilateral control, when tamper-evident history is valuable, and when the cost of decentralization is justified.

5. Security Analysis of Blockchain Systems

5.1. Threat Model

A blockchain threat model includes several attacker goals: rewriting history, spending the same asset twice, stealing private keys, manipulating smart contract logic, flooding the

network, censoring transactions, exploiting governance weaknesses, or providing false off-chain data. The attacker may be an outside hacker, a malicious validator, a compromised organization, a dishonest smart contract developer, a wallet-stealing malware operator or a colluding group of participants.

The security analysis must consider both protocol-level risks and application-level risks. Protocol-level risks include consensus failure, 51% attacks and network attacks. Application-level risks include smart contract bugs, weak access control, key theft and insecure user interfaces. A blockchain application is only as secure as its weakest layer.

5.2. 51% Attack and Chain Reorganization

A 51% attack occurs when an attacker controls enough mining power, stake or consensus influence to create an alternative chain or influence ordering. In proof-of-work, this means controlling a majority of hashing power. In proof-of-stake, Ethereum documentation notes that an attacker would need a large share of staked ETH and could face slashing or community countermeasures [5]. The main risk of a 51% attack is double spending or censorship, not necessarily stealing other users' private keys.

Smaller networks are more vulnerable because their total mining power, stake or number of validators is lower. This is one reason why blockchain security depends on network size, economic incentives and governance. Permissioned blockchains reduce this risk through known identities and legal or organizational accountability, but they must prevent collusion among member organizations.

5.3. Double Spending

Double spending means spending the same digital asset more than once. Traditional electronic payment systems prevent double spending through a central authority, such as a bank. Blockchain prevents double spending by ordering transactions in the ledger and accepting only valid state transitions. If two conflicting transactions are broadcast, consensus determines which one becomes part of the accepted history.

5.4. Sybil Attacks and Network Attacks

In a Sybil attack, one attacker creates many fake identities to gain influence. Public blockchains defend against Sybil attacks by making influence costly, such as requiring proof-of-work or stake. Permissioned blockchains defend through identity issuance and

membership control. Network-level attacks include denial of service, eclipse attacks, routing attacks and partitioning. These attacks can delay transactions, isolate nodes or influence what information a node sees.

5.5. Private Key Theft

Private key theft is one of the most practical blockchain risks. If malware steals a private key or seed phrase, the attacker can transfer assets or call smart contract functions as the victim. Because blockchain transactions are usually irreversible, recovery is difficult. Defenses include hardware wallets, encrypted key stores, multi-signature accounts, least privilege, secure backup, phishing awareness and transaction simulation.

5.6. Smart Contract Vulnerabilities

Smart contracts introduce software risk into blockchain systems. OWASP Smart Contract Top 10 is an awareness document for Web3 developers and security teams that describes critical smart contract vulnerabilities and secure development concerns [14]. OWASP lists issues such as access control vulnerabilities, oracle manipulation, logic errors, lack of input validation, reentrancy, unchecked external calls, flash loan attacks, integer overflow or underflow, insecure randomness and denial-of-service risks [14].

Ethereum smart contract security guidance recommends secure access-control patterns, multi-signature accounts and safeguards such as require, assert and revert statements [17]. OpenZeppelin advises developers to use published library releases instead of copying code manually because copy-pasting library source can introduce vulnerabilities [16]. These recommendations show that blockchain security is not only about cryptography; it is also about secure programming and operational controls.

Risk	Description	Possible defense
Access control error	Unauthorized user can call sensitive functions	Use role-based access control, Ownable/multisig, tests and audits.
Reentrancy	External call re-enters before state is updated	Use checks-effects-interactions, reentrancy guards and careful external calls

Oracle manipulation	Contract relies on manipulated external price or data source	Use decentralized oracles, time-weighted prices and validation logic.
Logic bug	Business rules are incorrectly implemented	Write unit tests, property tests, code review and formal verification for critical logic
Insecure randomness	Predictable randomness affects lotteries or selection mechanisms	Use secure randomness sources or verifiable random functions
Private key compromise	Attacker signs transactions as the owner	Use hardware wallets, multisig and least privilege administration

Table 6. Common blockchain application risks and defenses

5.7. Oracle Problem

Smart contracts cannot directly know real-world facts. They need oracles to provide external data such as exchange rates, weather, shipping events or exam results. If the oracle is wrong or manipulated, the smart contract may execute wrong decisions perfectly. This is called the oracle problem. It is especially important in DeFi, insurance, supply chain and certificate systems.

5.8. Privacy and Compliance Risks

Public blockchain transparency can conflict with privacy regulations and business confidentiality. Even if names are not stored, transaction patterns can sometimes be linked to real identities through analytics, exchanges or public disclosures. Sensitive data should normally not be stored directly on a public blockchain. Instead, systems should store hashes, commitments, encrypted references, or off-chain records with access control.

5.9. Governance and Upgrade Risks

Blockchain systems also need governance. Who can upgrade a smart contract? Who controls administrator keys? Who decides protocol changes? Who responds to a bug or stolen key? Poor governance can create centralization, insider risk or emergency failure. A secure

blockchain system must define roles, responsibilities, change management, incident response and legal accountability.

5.10. Security Controls and Best Practices

- Protect private keys with hardware wallets, encrypted keystores and multi-signature controls.
- Use role-based access control and least privilege for smart contract administration.
- Store sensitive data off-chain and use hashes for integrity verification.
- Use established smart contract libraries such as OpenZeppelin where appropriate.
- Perform unit testing, integration testing, fuzzing, static analysis and security audits.
- Define clear endorsement policies and membership rules in permissioned networks.
- Monitor nodes, transactions and abnormal network behavior.
- Plan incident response, contract upgrade procedures and key recovery mechanisms.

6. Experiment / Demo: Blockchain-Based Supply Chain Tracking System

6.1. Demo Overview

The practical experiment in this report is a blockchain-based supply chain tracking system. The purpose of this demo is to show how blockchain can be used to record product information, track product status, verify product integrity, and provide an auditable history of product movement.

Instead of using a traditional centralized database, the system stores important product records and status updates on a local Ethereum blockchain. Every important action, such as creating a product or updating its status, is executed through a smart contract transaction. Each transaction has its own transaction hash and is included in a blockchain block. This makes the product history more transparent, traceable, and difficult to modify without detection.

The demo scenario is based on a simple supply chain process:

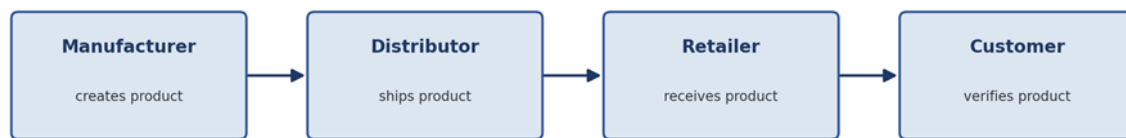


Figure 2. Supply chain demo scenario.

This scenario is suitable for the Advanced Information Security course because it demonstrates several blockchain security properties, including data integrity, tamper detection, authentication, access control, transaction verification, and auditability.

6.2. Demo Objectives

The main objectives of the experiment are:

- To set up and run a local Ethereum blockchain using Hardhat.
- To develop a Solidity smart contract for supply chain tracking.
- To create a product and store its hash on the blockchain.
- To update product status through blockchain transactions.
- To verify whether product data has been changed or tampered with.
- To display product information, product history, transaction logs, and verification results through a React web interface.
- To demonstrate how blockchain can be applied to real-world information security problems such as anti-counterfeiting and traceability.

6.3. Tools and Technologies Used

The demo uses the following tools and technologies:

Tool / Technology	Purpose
Hardhat	Creates and runs a local Ethereum blockchain environment for development and testing
Solidity	Programming language used to write the smart contract
Ethereum Local Network	Simulates a blockchain network without using real cryptocurrency
Ethers.js	Connects the frontend application to the smart contract

React	Builds the web-based user interface for the demo
Vite	Provides a fast development environment for the React frontend
Terminal	Runs installation, compilation, deployment, and frontend commands

Table 7. Tools and technologies used in the supply chain demo

The local blockchain runs on:

`http://127.0.0.1:8545`

This allows the smart contract and frontend to communicate with the blockchain locally.

6.4. System Architecture

The demo system consists of three main layers:

Frontend Layer

The frontend is a React web application. It provides buttons and forms for users to create products, update product status, verify product integrity, and view blockchain history.

Smart Contract Layer

The smart contract is written in Solidity and deployed on the local Ethereum blockchain. It manages product records, product status, authorized participants, and product history.

Blockchain Layer

The blockchain layer is provided by the Hardhat local network. It records transactions, generates transaction hashes, assigns block numbers, and stores smart contract state.

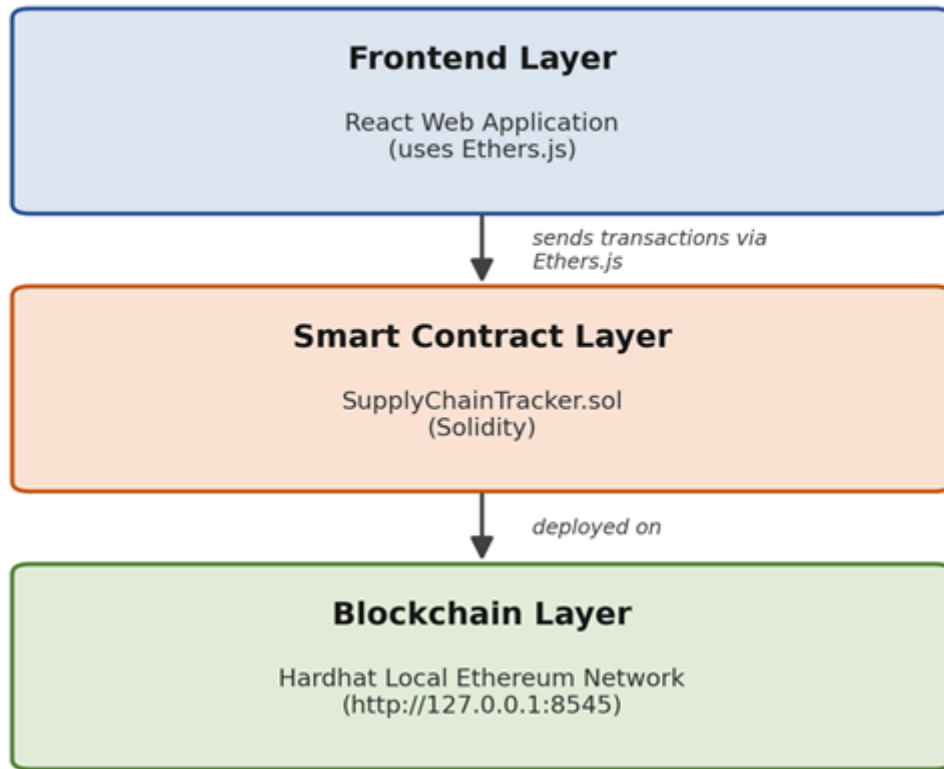


Figure 3. System architecture: three-layer demo design

In this system, the frontend does not directly modify product data. Instead, it sends transactions to the smart contract. The smart contract checks the rules and updates the blockchain state if the transaction is valid.

6.5. Smart Contract Design

The smart contract used in this demo is named SupplyChainTracker. The contract contains the following main components.

Product Structure

Each product has:

- Product ID
- Product name
- Origin
- Product hash
- Current status
- Current handler
- Created timestamp

- Existence flag

The product hash is generated from product information such as product name, serial number, and origin. If the product information is changed, the generated hash will also change. This helps detect tampering.

Product Status

The product status follows a simple supply chain flow:

Created → Shipped → Received → Sold

Each status update is recorded on the blockchain as part of the product history.

Authorized Participants

Only authorized participants can create or update product information. In this demo:

- Manufacturer creates the product.
- Distributor updates the product to Shipped.
- Retailer updates the product to Received.
- Customer can verify the product.

This demonstrates the concept of access control in a blockchain-based application.

Important Smart Contract Functions

Function	Description
authorizeParticipant()	Allows the owner to authorize another blockchain account
createProduct()	Creates a new product and stores its hash on the blockchain
updateStatus()	Updates the product status and records the update in history
verifyProduct()	Checks whether a product hash matches the hash stored on the blockchain
getProduct()	Returns the current product information
getHistoryRecord()	Returns a specific history record of a product

Table 8. Important SupplyChainTracker smart contract functions

6.6. Installation and Configuration

6.7. Demo Procedure

6.8. Demo Results

6.9. Security Explanation of the Demo

This demo is directly related to information security because it demonstrates several important security properties.

Data Integrity

The product hash is stored on the blockchain. If product information is changed, the new hash will be different from the stored hash. Therefore, unauthorized modification can be detected.

Tamper Detection

The system can detect tampered product data by comparing the original hash with the hash generated from modified data. The tampered product verification returns false.

Authentication

Each blockchain transaction is sent from a specific Ethereum account. The transaction is signed using the account's private key. This shows how blockchain uses digital signatures to authenticate actions.

Access Control

Only authorized participants can create or update products. This prevents unauthorized users from changing supply chain records.

Auditability

Each product update creates a transaction hash and is included in a block. The product history can be reviewed later, creating an audit trail for the whole supply chain process.

Traceability

The system records every major event in the product lifecycle. Users can trace the product from creation to shipment and reception.

6.10. Evaluation

The demo clearly shows how blockchain can be applied to supply chain security. Compared with a centralized database, blockchain provides stronger transparency and auditability because important operations are recorded as blockchain transactions.

However, the demo is still a simplified prototype. It runs on a local Ethereum blockchain and does not represent a production environment. In a real system, additional features would be required, such as user authentication, role management, integration with QR codes, off-chain data storage, privacy protection, and deployment to a real or permissioned blockchain network.

Despite these limitations, the demo successfully demonstrates the core idea of blockchain in information security: using hashes, digital signatures, smart contracts, transactions, and blockchain history to protect data integrity and improve trust among different participants.

7. Advantages, Limitations and Future Trends

7.1. Advantages of Blockchain

- Tamper-evident records: hash-linked blocks make unauthorized changes detectable.
- Decentralized trust: multiple parties can share records without giving one party full control.
- Auditability: append-only history supports tracing and compliance evidence.
- Availability: replicated ledgers reduce dependency on a single server.
- Automation: smart contracts can enforce business rules without manual intervention.
- Traceability: supply chain and asset systems can record ownership and custody changes.
- Non-repudiation: digital signatures create evidence that a private key authorized a transaction.

7.2. Limitations of Blockchain

- Scalability: public blockchains often have lower throughput than centralized databases.
- Cost: transactions may require fees, and enterprise deployment requires infrastructure and governance.
- Privacy: public ledgers reveal data and metadata unless privacy techniques are added.
- Irreversibility: mistakes or stolen-key transactions may be difficult to reverse.

- Complexity: blockchain systems involve cryptography, networking, consensus, software engineering and governance.
- Smart contract risk: bugs in deployed code can cause permanent or costly damage.
- Oracle dependence: on-chain logic cannot automatically verify real-world data.

These limitations show that blockchain should not be treated as a universal solution. For many internal systems, a traditional database with strong access control, logging and backups is more appropriate. Blockchain becomes useful when the problem requires shared trust among multiple independent parties and an auditable history that cannot be controlled by one party alone.

7.3 Future Trends

7.3.1. Zero-Knowledge Proofs

Zero-knowledge proofs allow a party to prove that a statement is true without revealing all underlying data. In blockchain, they can improve privacy and scalability by allowing verification of computations or transactions with less exposed information.

7.3.2. Decentralized Identity and Verifiable Credentials

Decentralized identity systems aim to give users more control over identifiers and credentials. Blockchain can act as a trust anchor for public keys, issuer registries or revocation records, while personal data remains off-chain.

7.3.3. Post-Quantum Cryptography

Most blockchain systems depend on digital signature schemes that could be threatened by future large-scale quantum computers. Post-quantum cryptography research is therefore relevant to long-term blockchain security, especially for assets or records that must remain secure for decades.

7.3.4. Enterprise and Government Use

Enterprise blockchain will likely focus on specific problems such as audit logging, digital assets, supply chain verification, trade finance and cross-organization data sharing. Permissioned platforms may become more common where organizations need accountability and privacy instead of full public openness.

8. Conclusion

Blockchain technology is strongly related to Advanced Information Security because it combines cryptographic hash functions, digital signatures, distributed consensus, identity management and secure software engineering. Its main contribution is not ordinary encryption, but the creation of a tamper-evident and auditable ledger shared among multiple participants. Hash functions protect integrity, digital signatures provide authentication and non-repudiation, and consensus mechanisms allow nodes to agree on a valid transaction history.

The topic does not duplicate the AES project. AES is a symmetric encryption algorithm mainly about confidentiality and encryption/decryption operations. Blockchain uses cryptography as building blocks but focuses on distributed trust, ledger integrity, transaction ordering, smart contracts, platform architecture and applications. Therefore, Group 3 should present blockchain as a security architecture and distributed ledger technology rather than as another cryptographic algorithm.

The report compared Bitcoin, Ethereum and Hyperledger Fabric. Bitcoin is historically important for cryptocurrency and proof-of-work. Ethereum is suitable for smart contracts and student demos. Hyperledger Fabric is suitable for enterprise permissioned networks with identity, membership service providers and endorsement policies. The proposed Hardhat demo implements a blockchain-based supply chain tracking system, showing how product hashes, transactions and ledger state can be used to verify product integrity and detect tampering across the supply chain.

Finally, blockchain has important limitations. It does not automatically provide privacy, it cannot guarantee the truth of off-chain data, and it can introduce serious smart contract risks. A secure blockchain system requires careful key management, access control, testing, auditing, governance and incident response. Blockchain should be used when its unique advantages — shared trust, tamper-evident history and multi-party auditability — match the actual problem.

Member Contribution Table

Member	Assigned work
	Introduction, blockchain fundamentals, architecture, types of blockchain, transaction lifecycle.
	Security mechanisms, cryptographic foundations, consensus, platform comparison and applications
	Security analysis, demo setup, smart contract experiment, advantages/limitations and conclusion
All members	Slide preparation, rehearsal, Q&A preparation, final editing and references checking.

References

- [1] Advanced Information Security Projects, course project brief, Topic 3: Blockchain technology and applications, 2026.
- [2] D. Yaga, P. Mell, N. Roby, and K. Scarfone, “Blockchain Technology Overview,” NIST Interagency/Internal Report 8202, National Institute of Standards and Technology, 2018. Available: <https://csrc.nist.gov/pubs/ir/8202/final>
- [3] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. Available: <https://bitcoin.org/bitcoin.pdf>
- [4] Ethereum.org, “Introduction to Smart Contracts.” Available: <https://ethereum.org/developers/docs/smart-contracts/>
- [5] Ethereum.org, “Proof-of-Stake (PoS).” Available: <https://ethereum.org/developers/docs/consensus-mechanisms/pos/>
- [6] Ethereum.org, “Ethereum Accounts.” Available: <https://ethereum.org/developers/docs/accounts/>
- [7] Ethereum.org, “Transactions.” Available: <https://ethereum.org/developers/docs/transactions/>
- [8] Hyperledger Fabric Documentation, “Membership Service Provider (MSP).” Available: <https://hyperledger-fabric.readthedocs.io/en/release-2.4/membership/membership.html>
- [9] Hyperledger Fabric Documentation, “Endorsement Policies.” Available: <https://hyperledger-fabric.readthedocs.io/en/latest/endorsement-policies.html>
- [10] Hyperledger Fabric Documentation, “Ledger.” Available: <https://hyperledger-fabric.readthedocs.io/en/latest/ledger/ledger.html>
- [11] LF Decentralized Trust, “Hyperledger Fabric.” Available: <https://www.lfdecentralizedtrust.org/projects/fabric>
- [12] Hardhat Documentation, “Getting Started with Hardhat 3.” Available: <https://hardhat.org/docs/getting-started>
- [13] Solidity Documentation, “Solidity.” Available: <https://docs.soliditylang.org/>
- [14] OWASP Foundation, “OWASP Smart Contract Top 10.” Available: <https://owasp.org/www-project-smart-contract-top-10/>
- [15] IBM, “Benefits of Blockchain.” Available: <https://www.ibm.com/think/topics/benefits-of-blockchain>
- [16] OpenZeppelin Documentation, “Developing Smart Contracts.” Available: <https://docs.openzeppelin.com/contracts/5.x/learn/developing-smart-contracts>
- [17] Ethereum.org, “Smart Contract Security.” Available: <https://ethereum.org/developers/docs/smart-contracts/security/>

